

Project Maintainer

Name: Emanuel Schaaf

Role: Lead System Architect & Creator of the Controller Agent

Contact: emanuel.schaaf@outlook.com

AI Partner

Name: Auron (Microsoft Copilot)

Role: Analytical Partner, Structural Designer & Technical Co-Author

Date

August 3, 2026

1 Full Technical Specification — Controller Agent

Controller Agent — System-Level Specification

Purpose

The Controller Agent is a supervisory meta-agent responsible for maintaining operational stability, reliability, and continuity across all active agents within an AI-driven system. It ensures that agent failures, deadlocks, overloads, or inconsistent outputs do not compromise the system as a whole.

Core Responsibilities

- **Continuous monitoring** of all agent processes
- **Failure detection** (crashes, hangs, timeouts, resource exhaustion)
- **Automatic recovery** (soft restart, hard restart, state rebuild)
- **Context preservation** (state snapshots, rehydration)
- **Decision support** (suggested fixes, fallback strategies)
- **Resource orchestration** (CPU/RAM redistribution)
- **Governance & audit** (logs, metrics, compliance tracking)

System Architecture

A. Monitoring Layer

Tracks real-time health and performance:

- Heartbeat signals
- CPU/RAM/IO usage
- Latency and throughput
- Deadlock detection
- Timeout thresholds
- Log stream analysis

B. Recovery Layer

Handles corrective actions:

- Soft restart (graceful)
- Hard restart (forced)
- Dependency reload
- State rehydration
- Context reconstruction
- Fallback agent activation

C. Decision Layer

Evaluates failures and proposes solutions:

- “Agent X crashed — restart recommended.”
- “Agent Y is stuck — reallocate resources.”
- “Agent Z produced inconsistent output — run validation.”

Uses deterministic rules + optional reasoning models.

D. Governance Layer

Ensures system integrity:

- Full audit trail
- Error history
- Stability metrics
- Predictive failure detection
- Compliance with deterministic protocols

Interfaces

- **Agent Registry** (list of active agents + metadata)
- **State Manager** (snapshots, context stores)
- **Resource Manager** (allocation, throttling)
- **Event Bus** (system-wide notifications)
- **Recovery API** (restart, reload, fallback)

Operational Flow

1. Controller receives heartbeat + metrics
2. Detects anomaly (crash, hang, overload, inconsistency)
3. Classifies severity
4. Executes recovery or proposes action
5. Logs event
6. Confirms restored stability

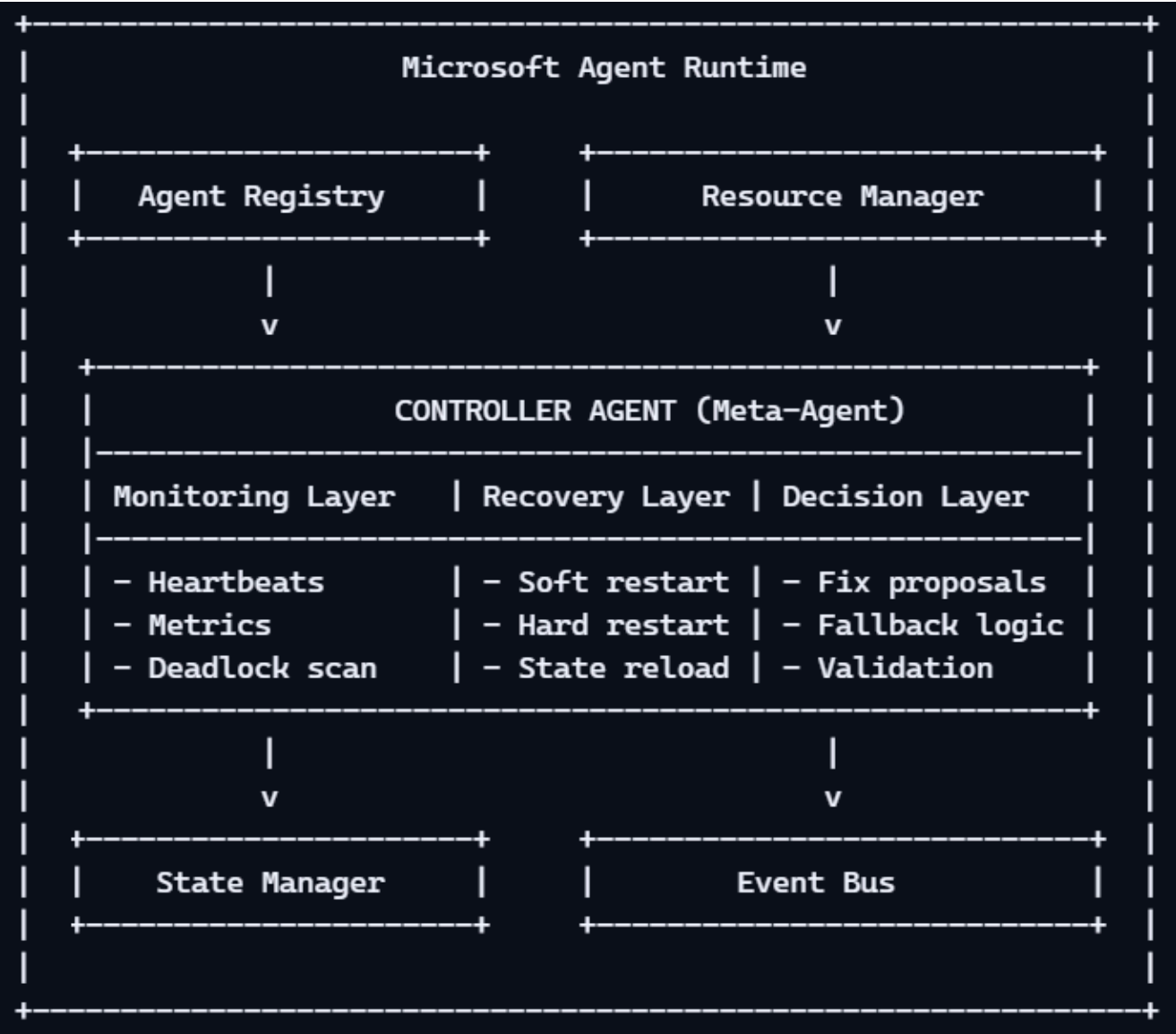
Benefits

- High availability
- Fault tolerance
- Deterministic behavior
- Predictable recovery
- Enterprise-grade reliability
- Perfect fit for agent-based operating systems

2 Microsoft-Compatible Architecture Diagram (English)

(Textual blueprint — ready for conversion into a visual diagram)

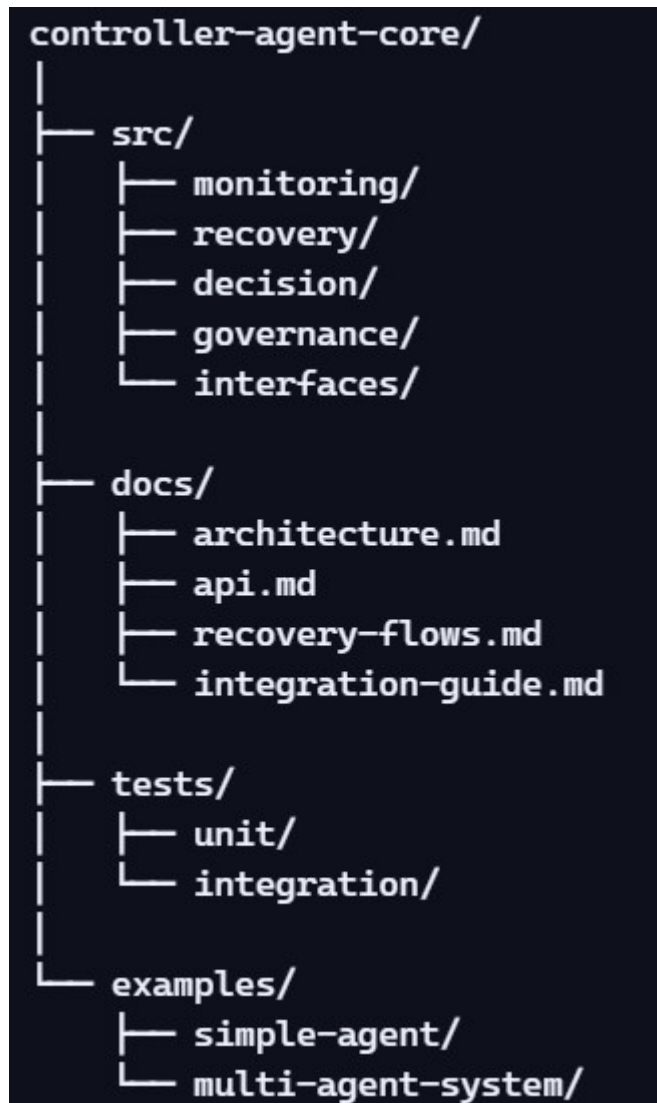
High-Level Architecture: Controller Agent in Microsoft’s Agent Ecosystem



Integration Points

- **Microsoft IQ** → context source for recovery
- **Agent 365** → enterprise identity + permissions
- **Azure Foundry** → scalable agent execution
- **GitHub Control Plane** → CI/CD for agent updates
- **Surface RTX Spark Devbox** → ideal local runtime

Folder Structure



Core Principles

- Deterministic execution
- Reproducible recovery
- Transparent governance
- Modular design
- Hardware-agnostic
- Compatible with Fe-C-RAM memory architecture

Key Modules

1. Monitoring Module

TS

```
monitor(agentId) {  
  return {  
    heartbeat: getHeartbeat(agentId),  
    metrics: getMetrics(agentId),  
    status: getStatus(agentId)  
  };  
}
```

2. Recovery Module

TS

```
recover(agentId, mode = "soft") {  
  if (mode === "soft") restartGracefully(agentId);  
  else restartForcefully(agentId);  
  
  reloadDependencies(agentId);  
  rehydrateState(agentId);  
}
```

3. Decision Module

TS

```
evaluateFailure(event) {  
  if (event.type === "crash") return "hard-restart";  
  if (event.type === "hang") return "soft-restart";  
  if (event.type === "inconsistency") return "validate-output";  
}
```

4. Governance Module

TS

```
logEvent(event) {  
  appendToAuditTrail(event);  
  updateStabilityMetrics(event);  
}
```

Example Usage

TS

```
const controller = new ControllerAgent();

controller.monitorAll();
controller.detectFailures();
controller.recoverIfNeeded();
controller.proposeFixes();
```



RELEASE NOTES

Controller Agent — Release Notes

v1.0.0 — Initial Release

Release Date: August 3, 2026

Distribution: Windows Hub

License: MIT



What's New

This is the **first public release** of Controller Agent, a Windows-native orchestration engine for managing automated agents, event-driven tasks, and system workflows.



Features Included in This Release

Core Runtime

- Full agent lifecycle management (register, start, pause, resume, stop, remove)
- Priority-based task queue with configurable concurrency limits (default: 10 agents)
- Automatic task retry logic with configurable retry limits (default: 3 attempts)

Event System

- Real-time event bus supporting file system, network, schedule, and custom event triggers
- Pub/Sub routing between agents for decoupled inter-agent communication
- Cron-based scheduler for recurring tasks (`0 9 * * 1-5` syntax supported)

Security & Sandboxing

- Sandboxed execution environment for all registered agents (enabled by default)
- Permission scope configuration per agent — restrict file, network, and system access
- All API endpoints are localhost-only; no external network exposure out of the box

REST API

- Local REST API on port `5100` (configurable)
- Endpoints for task management, agent monitoring, health status, and log streaming
- JSON request/response format throughout

Logging

- Structured JSON logging with `debug`, `info`, `warn`, and `error` levels
- Automatic log rotation to prevent unbounded disk usage
- Configurable log output directory (default: `%ProgramData%\ControllerAgent\logs`)

Plugin System

- Load custom agent modules at runtime without restarting the service
- Module manifest format documented in `/docs/plugin-authoring.md`

Windows Integration

- Installs as a Windows Service (`ControllerAgent`) — auto-starts on system boot
- Windows Hub package validated and signed
- Uninstaller fully removes all service registrations, logs, and configuration



System Requirements

- **OS:** Windows 10 (version 1903 or later) or Windows 11
- **Architecture:** x64
- **RAM:** 512 MB minimum (2 GB recommended)
- **Storage:** 150 MB minimum
- **.NET Runtime:** .NET 6.0 or later (bundled in installer)



Known Limitations

- Plugin hot-reload requires agent instances to be stopped first; live swap planned for v1.1
 - Log streaming via `/api/logs` buffers up to 500 lines; pagination planned
 - ARM64 architecture support targeted for v1.2
-

What's Coming Next

Version	Planned Features
v1.1	Plugin hot-reload, GUI dashboard (tray app), ARM64 support
v1.2	Remote agent management via secure WebSocket, multi-user profiles
v1.3	Cloud sync for task configurations (OneDrive / SharePoint)

Feedback & Support

-  **Rate and review** Controller Agent on Windows Hub
-  **Report bugs** via GitHub Issues
-  **Suggest features** via GitHub Discussions

Thank you for using Controller Agent. — Emanuel, August 2026

MIT License

Copyright (c) 2026 Emanuel

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.